CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

In colleges, faculty members are responsible for handling multiple academic and administrative activities such as lectures, laboratory sessions, meetings, mentoring students, and departmental tasks. Managing these activities efficiently requires a well-organized scheduling system. In many colleges, faculty schedules are maintained manually through printed timetables, spreadsheets, or informal communication. This approach can lead to various problems such as schedule conflicts, missed updates, lack of proper tracking, and delays in communication between faculty members and department administrators. As the number of faculty members and academic activities increases, managing schedules manually becomes difficult and inefficient.

The Faculty Day Planner is designed to provide a centralized and automated solution for managing faculty schedules and daily academic activities. The system allows faculty members to view their predefined weekly timetable and automatically generates a daily planner based on that timetable. Faculty members can easily check their lecture timings, lab sessions, and other academic responsibilities for the day. If there is a need to modify the schedule due to meetings, emergencies, or other departmental activities, the faculty member can submit a request for schedule modification. These requests are then reviewed and approved by the Head of the Department (HOD), ensuring that schedule changes are controlled and properly authorized.

The system also includes features such as attendance tracking, automated notifications, and schedule management. Whenever a schedule change is approved or updated, the system sends notifications to the respective faculty members to keep them informed. In addition, the system performs periodic maintenance tasks to remove temporary activities such as meetings or non-academic tasks from the timetable, ensuring that the core lecture and laboratory schedules remain consistent.

The application is implemented using the cloud-based platform Salesforce, which provides powerful tools for data management, workflow automation, approval processes, and secure storage. By using Salesforce features such as custom objects, flows, and approval workflows, the system ensures efficient schedule management and improved communication between faculty members and department administrators. Overall, the Faculty Day Planner helps institutions streamline faculty scheduling, reduce manual workload, and improve operational efficiency within the academic environment.

1.2 EXISTING SYSTEM

In colleges, faculty schedules are traditionally managed using manual methods such as printed timetables, spreadsheets, or simple document files. Each department usually prepares a weekly timetable that includes lecture hours, laboratory sessions, and other academic activities assigned to faculty members. These timetables are often distributed in printed format or shared through email or messaging platforms. Although this approach has been used for many years, it becomes increasingly difficult to manage as the number of faculty members, courses, and academic activities grows.

In the existing system, any changes to the schedule, such as class cancellations, additional meetings, or timetable adjustments, are usually communicated through emails, phone calls, or verbal messages. Faculty members must regularly check multiple sources of information to stay updated about their schedules. Sometimes changes are communicated at the last minute, which may lead to confusion or miscommunication among faculty members and department administrators.

Another challenge of the existing system is the lack of centralized data management. Since schedules are maintained in separate files or documents, it becomes difficult to track updates, maintain historical records, or verify the latest version of the timetable. In cases where a faculty member needs to modify their schedule due to a meeting or personal reason, they must inform the Head of the Department manually, which may delay the approval process.

Additionally, attendance tracking and daily activity monitoring are not properly integrated with the scheduling system. Faculty members may need to maintain separate records for attendance or daily work, which increases the administrative workload. Due to the absence of automation, the manual system is prone to errors such as incorrect scheduling, overlapping classes, or missed updates.

Therefore, the existing system lacks efficiency, transparency, and real-time communication. These limitations highlight the need for a digital system that can centrally manage faculty schedules, automate approval processes, and provide real-time updates to faculty members and administrators.

1.3 DRAWBACKS OF EXISTING SYSTEM

The existing manual system has several limitations:
The main drawbacks are:

- Difficult to manage large numbers of faculty schedules.
- Lack of centralized schedule management.
- High chance of human errors.
- Schedule changes are not tracked properly.
- Communication delays between faculty and HOD.
- No automated approval system.
- Difficult to maintain daily activity records.

1.4 PROPOSED SYSTEM

The proposed system introduces a digital Faculty Day Planner developed using the cloud-based platform Salesforce. The system maintains a structured weekly timetable for each faculty member and automatically generates a daily planner based on the predefined schedule items. Faculty members can log into the application to view their daily academic activities such as lectures, laboratory sessions, and other assigned tasks. This centralized system ensures that all schedule information is stored and managed in a single platform, making it easier for faculty members to access their timetable and plan their daily work efficiently.

In this system, faculty members are also given the ability to request modifications to their schedule whenever necessary. For example, if a faculty member has an urgent meeting, seminar, or any unexpected responsibility, they can submit a schedule change request through the application. The request is then forwarded to the Head of the Department (HOD) for approval. The HOD reviews the request and decides whether to approve or reject the change. Once the request is approved, the schedule is automatically updated in the system, and notifications are sent to the concerned faculty member to keep them informed about the changes.

Another important feature of the proposed system is automation. The system includes scheduled processes that automatically manage temporary activities in the timetable. For instance, tasks such as meetings or special events that are added temporarily will be removed automatically during the weekly maintenance process, ensuring that only the regular lecture and laboratory sessions remain in the default timetable. This automation reduces manual work, maintains timetable accuracy, and ensures smooth academic scheduling within the department.

1.5 ADVANTAGES OF PROPOSED SYSTEM

The proposed Internship Application Management System offers several advantages over the existing manual system. The major advantages are listed below:

- **Centralized Schedule Management:** All faculty timetable details, daily planners, and schedule updates are stored in a single platform using Salesforce, making it easy to access and manage information.
- **Automated Approval Process:** The approval workflow enables Department Placement Officers, HOD, and College Placement Officers to review and approve applications efficiently.
- **Real-Time Notifications:** The system automatically sends email to faculty members whenever their schedules are updated or approved.
- **Reduced Manual Work and Errors:** Automation reduces the need for manual scheduling and minimizes the chances of timetable conflicts or human errors.
- **Improved Communication and Efficiency:** The system improves coordination between faculty and administrators by providing quick updates and organized schedule management.

SUMMARY

This chapter introduced the Faculty Day Planner developed using Salesforce. It explained the need for an efficient system to manage faculty schedules and daily academic activities. The chapter also described the existing manual system used in many institutions. The limitations and drawbacks of the current approach were discussed in detail. Finally, the proposed automated system and its advantages in improving schedule management and communication were presented.

CHAPTER 2

SYSTEM ANALYSIS

2.1 IDENTIFICATION ON NEED

In colleges, managing faculty schedules and daily academic activities is often done manually using printed timetables, spreadsheets, or informal communication methods. As the number of faculty members, courses, and departmental activities increases, it becomes difficult to maintain accurate and updated schedules. Faculty members may face issues such as timetable conflicts, sudden schedule changes, and lack of proper communication regarding meetings or academic responsibilities. These problems highlight the need for a structured and centralized system that can effectively manage faculty schedules.

The Faculty Day Planner addresses this need by providing a digital platform where faculty members can easily view their weekly timetable and daily schedule. The system also allows faculty to request schedule modifications when necessary, which are then reviewed and approved by the Head of the Department. By implementing this system using the cloud-based platform Salesforce, institutions can automate schedule management, improve communication between faculty and administrators, and ensure that academic activities are organized efficiently.

2.2 FEASIBILITY STUDY

Feasibility study is an important step in system development that determines whether a proposed system is practical and beneficial for an organization. It evaluates different aspects such as economic, operational, and technical feasibility to ensure that the system can be successfully implemented. The main objective of the feasibility study is to identify potential problems, required resources, and overall benefits before developing the system.

For the Faculty Day Planner, the feasibility study ensures that the application can effectively manage faculty schedules, daily planners, and approval processes in a college environment. The system is developed using the cloud-based platform Salesforce, which provides a reliable infrastructure for building and managing applications. The feasibility analysis confirms that the system can be implemented with minimal cost, simple operation, and available technical resources.

The study also examines whether the system will improve the efficiency of schedule management and communication between faculty members and the Head of the Department (HOD). By automating scheduling tasks, approval workflows, and notifications, the system reduces manual work and helps the institution maintain an organized academic schedule. Therefore, the proposed system is considered feasible and beneficial for the institution.

2.2.1 Economic Feasibility

Economic feasibility evaluates whether the proposed system is cost-effective and beneficial compared to the existing manual system. The Faculty Day Planner reduces the expenses associated with paper-based timetable management, manual record keeping, and repeated communication for schedule updates. By implementing the system on the cloud platform Salesforce, institutions do not need to invest heavily in physical infrastructure such as servers or additional hardware.

The system also minimizes administrative workload by automating tasks such as schedule generation, approval workflows, and notifications. This helps save time and operational costs for the institution. Since the platform provides built-in tools for data management, security, and automation, the development and maintenance costs are relatively low.

Therefore, the proposed system is economically feasible as it reduces manual effort, lowers operational costs, and improves the overall efficiency of schedule management within the institution.

2.2.2 Operational Feasibility

Operational feasibility evaluates whether the proposed system can be easily used and accepted by the users within the organization. The Faculty Day Planner is designed with a simple and user-friendly interface so that faculty members and administrators can operate it without difficulty. Faculty members can easily view their weekly timetable, check their daily planner, and submit schedule change requests through the system. Similarly, the Head of the Department (HOD) can review and approve these requests efficiently.

The system improves daily operations by automating schedule management and communication between faculty and administrators. Notifications are automatically sent whenever schedules are updated or approved, ensuring that users receive timely information. Since the system is developed using the cloud platform Salesforce, it can be accessed through a web browser from any location with internet connectivity.

Therefore, the system is operationally feasible because it simplifies schedule management, reduces manual work, and enhances coordination among faculty members and department administrators.

2.2.3 Technical Feasibility

Technical feasibility evaluates whether the required technology, tools, and resources are available to develop and implement the proposed system. The Faculty Day Planner is developed using the cloud-based platform Salesforce, which provides a reliable and scalable environment for building applications. Salesforce offers features such as custom objects, workflow automation, approval processes, and secure data storage, which are essential for implementing the functionalities of the system.

The system does not require complex hardware or specialized infrastructure because it operates on the cloud. Users only need a computer or mobile device with an internet connection and a web browser to access the application. Salesforce also provides built-in tools for data management, reporting, and user access control, which simplifies the development and maintenance process.

Therefore, the proposed system is technically feasible as the required technology, software tools, and resources are readily available. The platform supports secure data handling, easy integration, and efficient system performance, making it suitable for implementing the Faculty Day Planner.

2.3 SOFTWARE REQUIREMENT SPECIFICATIONS

Software Requirement Specifications (SRS) describe the software and hardware requirements needed to develop and run the proposed system. It provides a clear understanding of the tools, technologies, and resources required for the successful implementation of the Faculty Day Planner.

2.3.1 Software Requirements

The software requirements specify the applications and technologies needed to develop and operate the system. The main software used for developing the Faculty Day Planner includes:

- **Operating System:** Windows 10 or above
- **Platform:** Salesforce Platform
- **Development Environment:** Salesforce Developer Edition
- **Technology Used:** Salesforce Experience Cloud, Flows, Approval Process
- **Browser:** Google Chrome / Microsoft Edge
- **Documentation Tool:** Microsoft Word

These software tools help in designing, developing, and managing the Faculty Day Planner efficiently.

2.3.2 Hardware Requirements

Hardware requirements refer to the physical components needed to run the system smoothly. The minimum hardware requirements for the proposed system are:

- **Processor:** Intel Core i3 or above
- **RAM:** 4 GB or higher
- **Hard Disk:** 250 GB or higher
- **Display:** Standard monitor with internet connectivity
- **Input Devices:** Keyboard and Mouse

These hardware components are sufficient to develop, deploy, and use the Faculty Day Planner effectively.

2.4 SOFTWARE DESCRIPTION

2.4.1 Front End

The front end of the Faculty Day Planner represents the user interface through which faculty members and administrators interact with the application. It is responsible for displaying information, collecting user input, and providing easy navigation between different features of the system. The front end is designed to be simple, user-friendly, and visually organized so that users can easily access their schedules, daily planners, and other system functions without technical difficulty.

In this project, the front end is developed using the Lightning user interface provided by the cloud platform Salesforce. The interface includes various components such as dashboards, forms, record pages, and navigation menus that allow users to manage faculty schedules and daily activities efficiently. Faculty members can log in to the system to view their timetable, create or update day planner entries, and submit schedule change requests. Similarly, the Head of the Department (HOD) can access approval requests and manage schedule updates through the interface.

The Lightning interface also supports responsive design, which allows the application to be accessed from different devices such as computers, tablets, and mobile phones. This flexibility ensures that faculty members can check their schedules and receive notifications at any time. Overall, the front end provides an interactive and efficient platform for users to perform their tasks within the Faculty Day Planner.

2.4.2 Back End

The back end of the Faculty Day Planner is responsible for handling the internal processing of data, business logic, and system automation. It manages all operations such as storing schedule information, processing approval requests, generating daily planners, and sending notifications to users. The back end ensures that the system functions correctly and that all data entered by users is securely stored and processed.

In this project, the back end is implemented using the cloud platform Salesforce. Salesforce provides powerful tools such as custom objects, Flow Builder, approval processes, validation rules, and automation features to manage the system's functionality. These tools help automate tasks like creating day planner records from schedule items, sending email notifications to faculty members, and handling schedule change approvals by the Head of the Department (HOD).

The back end also manages user access and security by controlling permissions for faculty members and administrators. Data is stored securely within the Salesforce database, ensuring reliability and data protection. Overall, the back end plays a crucial role in maintaining system performance, processing user requests, and ensuring smooth operation of the Faculty Day Planner.

2.4.3 Database

The database is an important component of the Faculty Day Planner as it stores and manages all the data required for the application. It maintains information related to faculty members, schedules, day planners, attendance records, and approval requests. The database ensures that data is organized, easily accessible, and securely stored so that users can retrieve and update information whenever necessary.

In this project, the database is managed using the cloud-based platform Salesforce. Salesforce uses objects to store data, which function similarly to tables in a traditional database system. Custom objects such as Faculty, Schedule Item, Day Planner, and Attendance are created to store relevant information. Each object contains fields that represent specific data attributes, such as faculty name, schedule time, date, subject, and approval status.

The Salesforce database also supports relationships between objects, allowing data to be connected and organized efficiently. For example, the Day Planner object is linked to the Faculty and Schedule Item objects to track daily schedules. This structured database design ensures data integrity, improves data retrieval, and supports the smooth functioning of the Faculty Day Planner.

SUMMARY

This chapter discussed the system analysis and the important factors required for developing the Faculty Day Planner using Salesforce. It explained the need for the system and how it can improve the management of faculty schedules. The feasibility study analyzed the economic, operational, and technical aspects of the project. The chapter also described the hardware and software requirements necessary for implementation. Overall, this analysis ensures that the proposed system can be developed and used effectively within a colleges.

CHAPTER 3

SYSTEM DESIGN

3.1 MODULE DESCRIPTION

The Faculty Day Planner is divided into several modules to ensure proper organization and efficient functioning of the system. Each module performs specific tasks and manages different activities related to internship applications.

The following modules are included in the Faculty Day Planner:

- Faculty Management Module
- Schedule Item (Timetable) Management Module
- Day Planner Module
- Attendance Management Module
- Schedule Change Request & Approval Module
- Notification Module
- Weekly Schedule Maintenance Module
- Reports and Dashboard Module

3.1.1 Faculty Management Module:

This module is responsible for maintaining and managing the details of faculty members in the system. This module stores important information such as faculty name, department, designation, email address, and other relevant details. It acts as the central record for all faculty members who use the system and ensures that their information is organized and easily accessible.

Through this module, administrators can add, update, or manage faculty records whenever necessary. Each faculty member is associated with their respective schedule items and day planner entries, which allows the system to generate daily schedules automatically. This module also helps identify the faculty responsible for particular lectures, laboratory sessions, or other academic activities.

The module is implemented using custom objects and fields in the cloud platform Salesforce. By maintaining accurate faculty data, the system ensures proper scheduling, efficient management of academic tasks, and effective communication between faculty members and the Head of the Department.

3.1.2 Schedule Item Management Module:

This module is responsible for maintaining the default weekly timetable of faculty members. This module stores details such as day of the week, subject, time slot, class type (lecture or lab), and the faculty assigned to each session. It acts as the base structure for generating the daily planner for faculty members.

Through this module, administrators can create and manage schedule items for the five-day academic week. Each schedule item represents a specific teaching activity assigned to a faculty member. These timetable records are later used by the system to automatically generate daily planner entries for the corresponding day.

This module is implemented using custom objects and fields in the platform Salesforce. By maintaining a structured timetable in the system, the module ensures accurate schedule management and helps faculty members easily track their academic responsibilities.

3.1.3 Day Planner Module:

This module is responsible for generating and managing the daily schedule of faculty members. It creates a day-wise plan based on the predefined schedule items stored in the timetable. This module allows faculty members to view their daily academic activities such as lectures, laboratory sessions, and other assigned tasks in an organized manner.

When a new day planner record is created for a particular date, the system automatically copies the relevant schedule items from the weekly timetable. This helps faculty members quickly access their tasks for that specific day without manually entering schedule details. The module also allows faculty members to review their daily planner and request schedule modifications if necessary.

This module is implemented using automation tools and custom objects in the cloud platform Salesforce. By automatically generating daily schedules, the Day Planner Module simplifies schedule management and ensures that faculty members stay informed about their daily academic responsibilities.

3.1.4 Attendance Management Module:

This module is responsible for recording and maintaining the attendance details of faculty members. This module allows the system to track whether faculty members are present and actively conducting their scheduled academic activities. It helps maintain proper records of faculty attendance for administrative and monitoring purposes.

Through this module, attendance information is linked with the daily planner and schedule items of each faculty member. This ensures that attendance records correspond to the scheduled lectures or laboratory sessions assigned to the faculty. The stored attendance data can also be used for generating reports and analyzing faculty participation in academic activities.

The module is implemented using custom objects and automation features available in the cloud platform Salesforce. By maintaining accurate attendance records, the system supports better management of faculty activities and helps the department monitor academic responsibilities effectively.

3.1.5 Schedule Change Request & Approval Module:

This module allows faculty members to request modifications to their daily schedule when necessary. Sometimes faculty members may need to change their scheduled lectures or activities due to meetings, seminars, or other important tasks. Through this module, faculty members can submit a schedule change request directly in the system by providing the required details.

Once the request is submitted, it is automatically sent to the **Head of the Department (HOD)** for review. The HOD can examine the request and decide whether to approve or reject it based on the department's requirements and timetable availability. If the request is approved, the schedule is updated in the system accordingly. If it is rejected, the original schedule remains unchanged.

This module is implemented using approval workflows and automation tools available in the platform Salesforce. The approval process ensures that all schedule changes are properly authorized and recorded, which helps maintain transparency and avoid scheduling conflicts within the department.

3.1.6 Notification Module:

This module is responsible for sending alerts and updates to faculty members regarding their schedules and planner activities. This module ensures that users receive timely information whenever important actions occur in the system. Notifications are triggered when a day planner is created, when schedule items are updated, or when a schedule change request is approved or rejected by the Head of the Department.

The module helps improve communication between faculty members and administrators by automatically informing users about any changes to their schedules. Faculty members can quickly check their updated timetable and plan their activities accordingly without the need for manual communication.

This functionality is implemented using email alerts and automated workflows provided by the cloud platform Salesforce. By providing real-time notifications, the module ensures that faculty members remain informed about their daily schedules and any important updates in the system.

3.1.7 Weekly Schedule Maintenance Module:

This module is responsible for maintaining and cleaning the schedule data regularly. In the Faculty Day Planner, certain schedule items such as meetings, events, or other temporary activities may be added to the timetable during the week. These activities are not part of the regular academic schedule and therefore need to be removed periodically to maintain a clear and accurate timetable.

This module automatically performs a maintenance process every Monday morning. During this process, the system checks the schedule items and removes entries that are not classified as lecture or laboratory sessions. By doing this, the system ensures that only the regular teaching activities remain in the default timetable structure.

The module is implemented using scheduled automation features available in the cloud platform Salesforce. This automation reduces manual work for administrators and helps maintain a clean and organized schedule for faculty members.

3.1.8 Reports and Dashboard Module:

This module provides a visual and analytical view of the data stored in the Faculty Day Planner. This module helps administrators and the Head of the Department monitor faculty schedules, attendance records, and planner activities effectively. Reports present structured information such as the number of scheduled lectures, attendance details, and schedule changes in a clear format.

Dashboards display important information using charts, graphs, and summary panels, which make it easier to analyze faculty activities and departmental schedules. These visual representations help users quickly understand system data and make better decisions regarding schedule management.

This module is implemented using the reporting and dashboard tools available in the platform Salesforce. By providing detailed reports and visual insights, the module helps improve monitoring, planning, and overall management of faculty schedules within the department.

3.2 DATA FLOW DIAGRAM

A Data Flow Diagram (DFD) is a graphical representation that shows the flow of data through a system. It illustrates how data moves from one entity to another, how it is processed, and how it is stored. DFDs help in understanding the overall system functionality and interactions between external entities and internal processes.

3.2.1 DFD Level 0

DFD Level 0 provides a high-level overview of the system's data flow by representing the system as a single process and showing interactions with external entities. It focuses on major data exchanges without detailing internal processing steps.

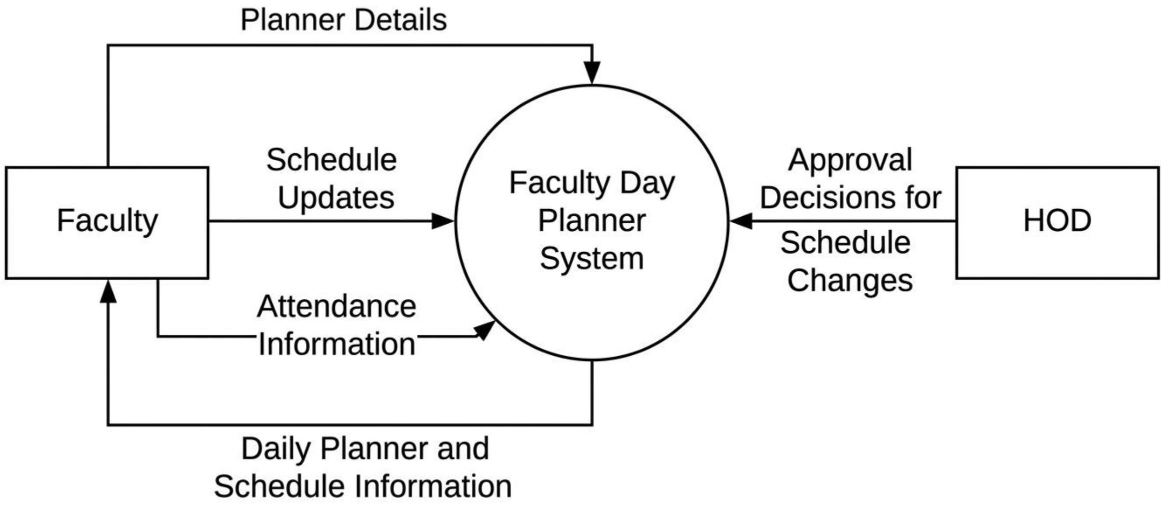

Fig 3.1 Data Flow Diagram Level 0

The DFD Level-0 (Context Diagram) represents the overall interaction between users and the Faculty Day Planner. In this type of diagram, the entire system is shown as a single process, while external users interact with it by sending or receiving data. The main purpose of a Level-0 DFD is to give a high-level overview of how information flows between external entities and the system without showing internal processing details.

In the diagram, two external entities are shown: Faculty and HOD (Head of the Department). Faculty members send planner details, schedule updates, and attendance information to the system. These inputs help the system manage daily schedules and maintain attendance records. The system processes this data and provides daily planner and schedule information back to the faculty so they can view their tasks and timetable for the day.

The HOD interacts with the system mainly for schedule change approvals. When a faculty member requests a schedule modification, the request is processed by the system and sent to the HOD for review. The HOD then sends approval decisions for schedule changes back to the system. Based on this decision, the system updates the schedule and informs the faculty. This diagram therefore shows the overall data flow between faculty, the HOD, and the Faculty Day Planner.

3.2.2 DFD Level 1

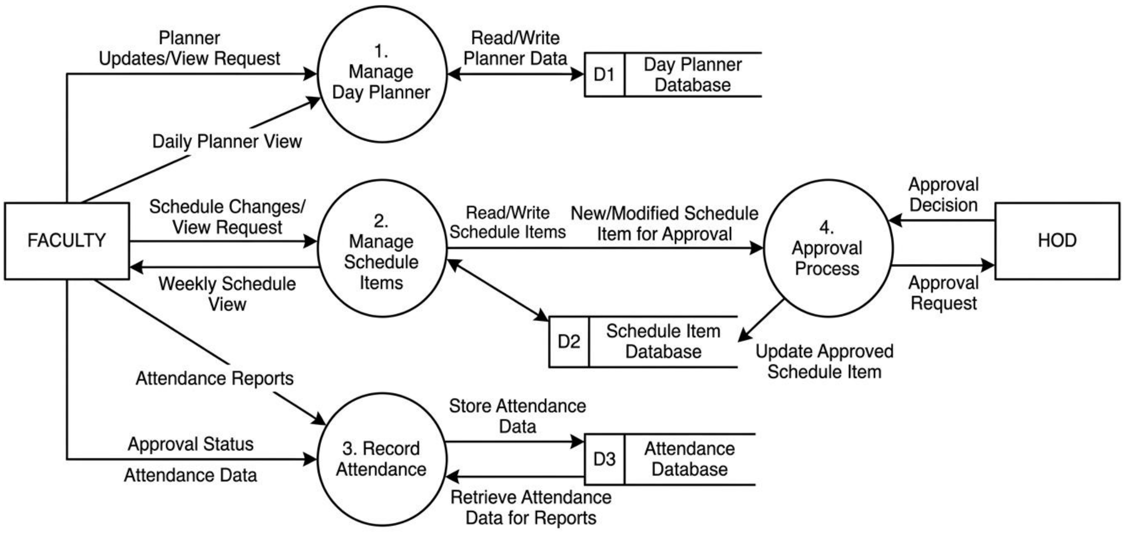

Fig 3.2 Data Flow Diagram Level 1

The DFD Level-1 diagram provides a detailed view of the Faculty Day Planner by breaking the main system into several internal processes and showing how data moves between users, processes, and databases. A Level-1 DFD decomposes the main system into multiple subprocesses so that the internal functions and data flows can be understood clearly. It includes components such as external entities, processes, data flows, and data stores that represent how information is processed and stored in the system.

In this diagram, the Faculty interacts with three main processes: Manage Day Planner, Manage Schedule Items, and Record Attendance. Faculty members send planner updates and view requests to the Manage Day Planner process, which reads and writes planner data to the Day Planner Database (D1). Faculty can also view weekly schedules or request schedule changes through the Manage Schedule Items process, which stores and retrieves timetable information from the Schedule Item Database (D2). Additionally, faculty submit attendance data through the Record Attendance process, where the system stores attendance records in the Attendance Database (D3) and retrieves them to generate attendance reports.

Another important part of the diagram is the Approval Process, which handles schedule change requests. When a faculty member modifies or requests a change to a schedule item, the request is sent to the Approval Process. This process forwards the approval request to the HOD (Head of the Department), who reviews it and sends back an approval decision.

Based on the HOD's decision, the system updates the schedule item in the database and informs the faculty about the approval status. This Level-1 DFD therefore explains how the system manages daily planners, schedules, attendance records, and approval workflows in a structured manner.

3.3 SYSTEM FLOW DIAGRAM

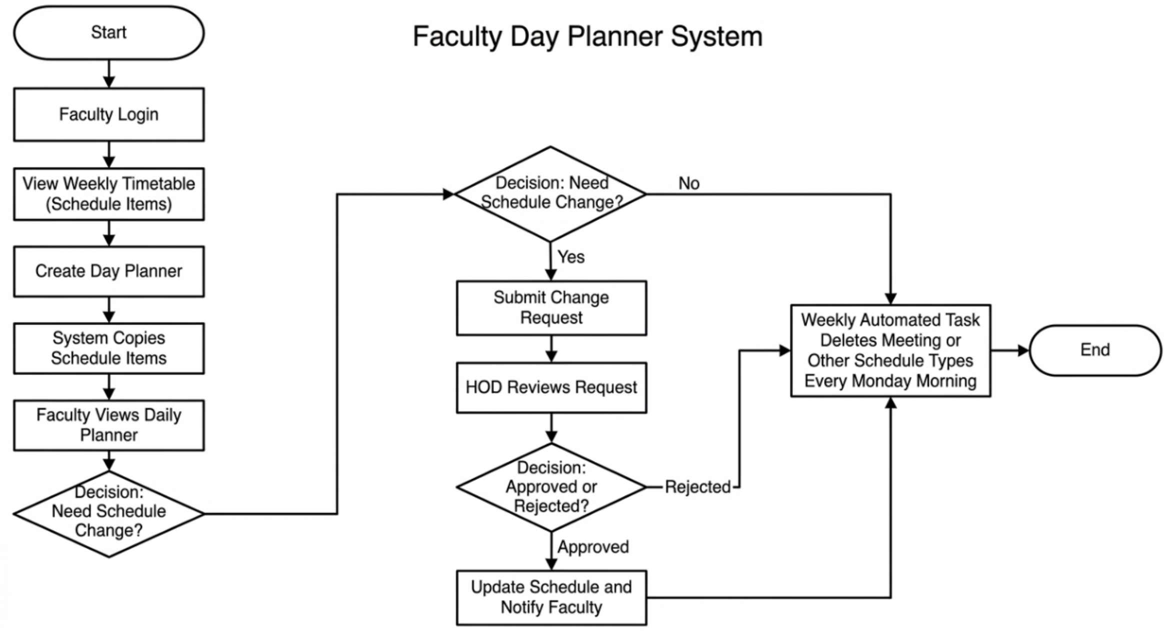

Fig 3.3 System Flow Diagram

The System Flow Diagram for the Faculty Day Planner represents the sequence of activities that occur from the moment a faculty member accesses the system until the completion of the process. A system flow diagram visually illustrates the steps, decisions, and interactions within a system using symbols such as rectangles for processes and diamonds for decisions. These diagrams help in understanding the workflow and logical sequence of operations in a system.

The process begins when the faculty member logs into the system and views the weekly timetable (schedule items). After reviewing the timetable, the faculty creates a day planner, and the system automatically copies the relevant schedule items into the planner for that particular day. The faculty then views the generated daily planner to understand their tasks and teaching schedule. At this stage, the system checks whether the faculty needs to make any schedule changes. If no changes are required, the process directly moves to the automated weekly maintenance task.

If the faculty needs to modify the schedule, they submit a schedule change request through the system. The request is then reviewed by the Head of the Department (HOD). The HOD decides whether to approve or reject the request. If the request is approved, the system updates the schedule and sends a notification to the faculty member. If the request is rejected, the existing schedule remains unchanged. Finally, the system performs a weekly automated task every Monday morning, which removes temporary schedule items such as meetings or other non-lecture activities, ensuring that only the regular timetable remains in the system before the process ends.

3.4 INPUT DESIGN

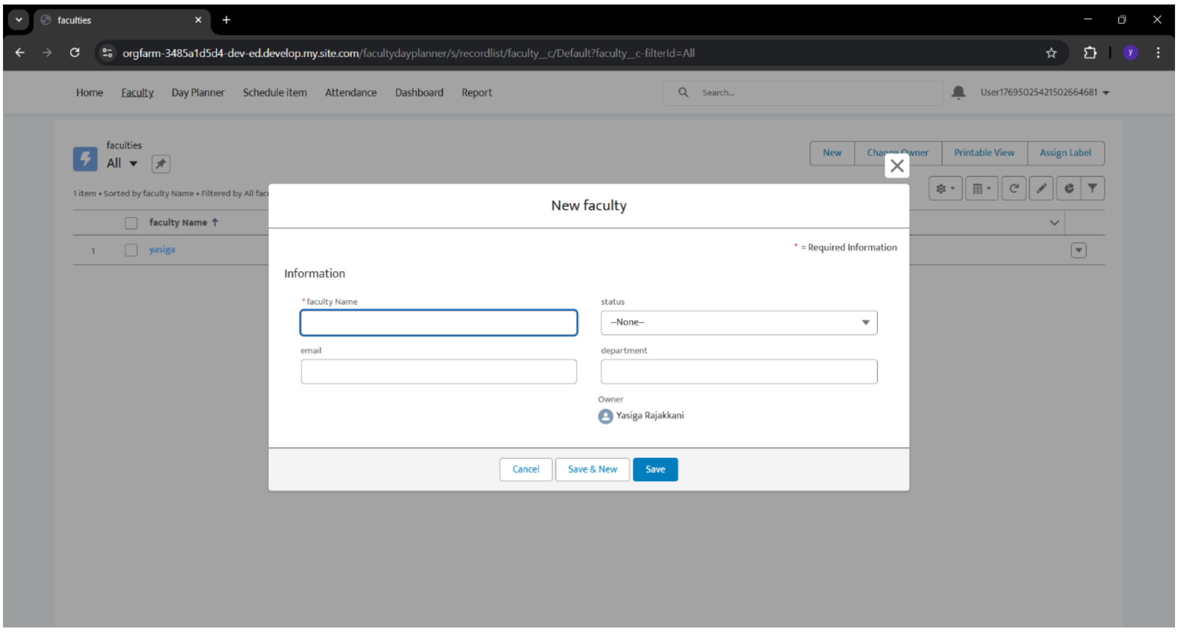

Figure 3.4 Faculty

This image shows the Faculty Input Form used in the Faculty Day Planner to enter new faculty information. Input design focuses on creating simple and user-friendly forms that allow users to enter data accurately into the system. The form contains fields such as Faculty Name, Email, Department, and Status for recording faculty details. Required fields are marked with an asterisk (*) to ensure important data is entered before saving. Well-designed input forms help reduce data entry errors and make the system easier to use.

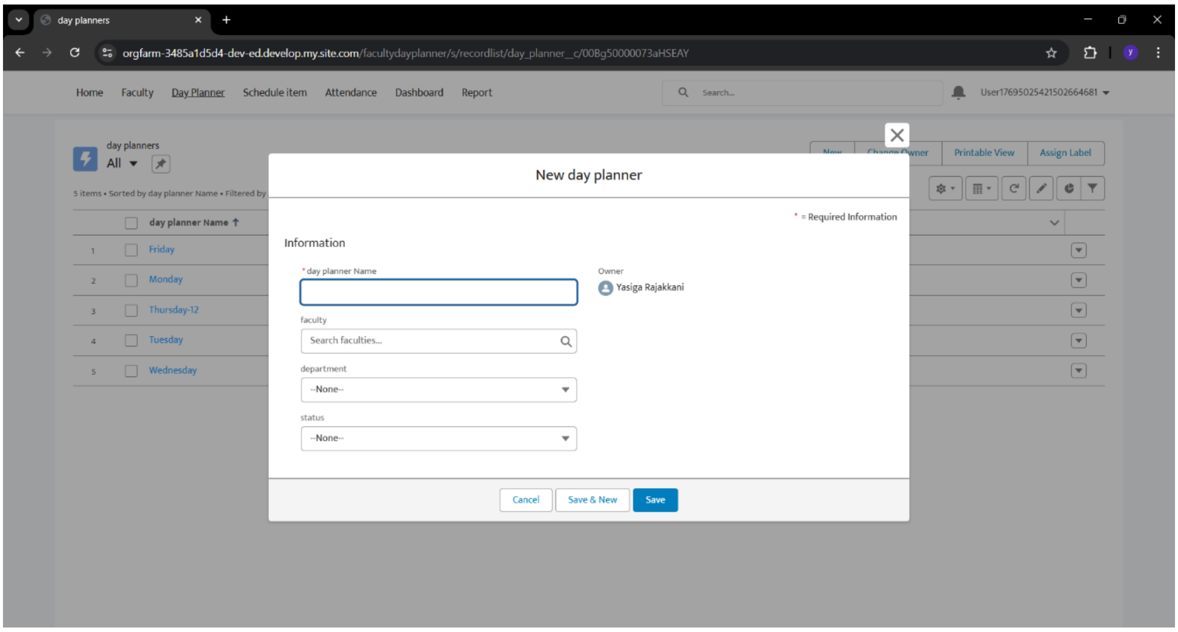

Figure 3.5 Day Planner

The image represents an input design form used to create a new day planner in the system, where users can enter details such as day planner name, faculty, department, and status through structured fields. It includes different input components like text boxes, dropdown menus, and search fields, which help in collecting accurate and organized data from users. Forms like this are essential for capturing user input efficiently and ensuring data is entered in a consistent format. The presence of clearly labeled fields and required indicators guides users to complete the form correctly, while action buttons like Save and Cancel provide control over data submission, making the interface simple, user-friendly, and effective for system interaction.

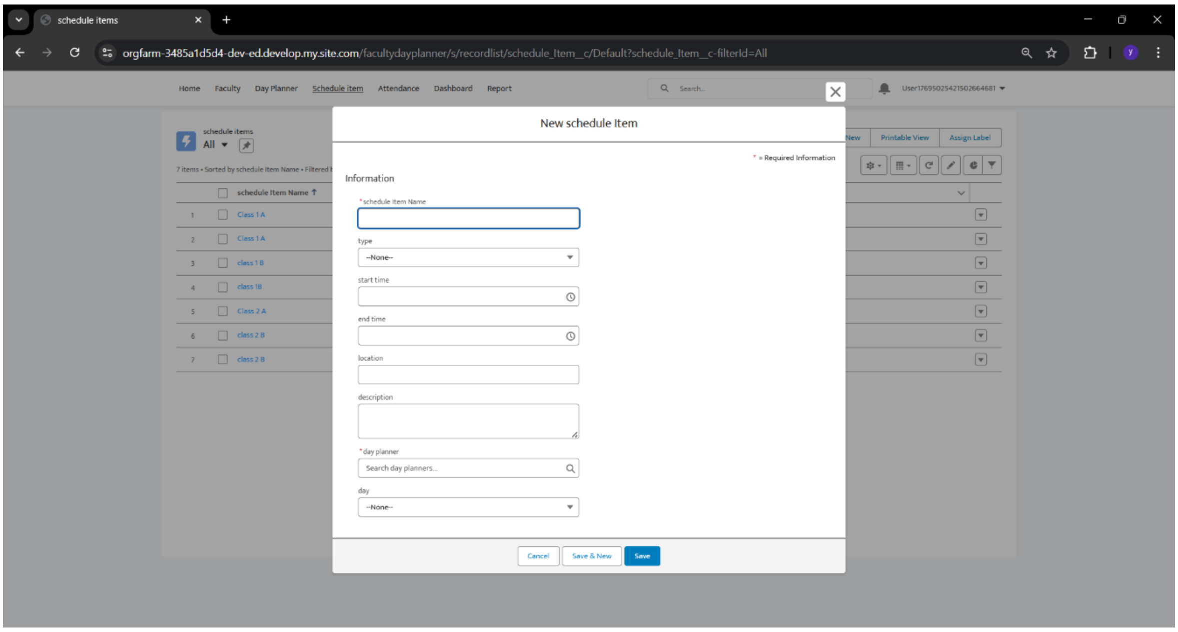

Figure 3.6 Schedule Item

The image represents an input design form for creating a new schedule item in the Faculty Day Planner Syste, where users enter details such as schedule item name, type, start time, end time, location, description, and associated day planner. The form uses structured input elements like text fields, dropdown menus, and time selectors to ensure accurate and organized data entry. Clearly labeled fields and required indicators guide users to provide complete information, reducing errors during input. Well-designed input forms like this improve data accuracy, usability, and efficiency by making data entry simple and consistent. Additionally, action buttons such as Save, Save & New, and Cancel enhance user control, making the interface intuitive and effective for system interaction.

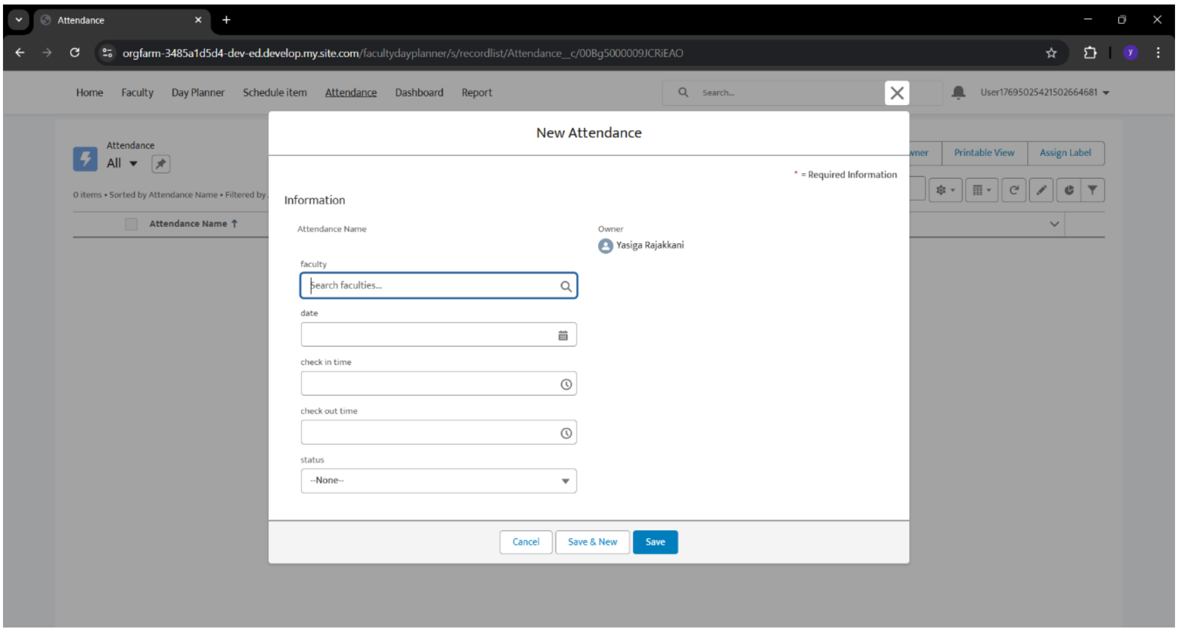

Figure 3.7 Attendance

The image represents an input design form for recording attendance details in the Faculty Day Planner, where users enter information such as faculty name, date, check-in time, check-out time, and status through structured fields. The form uses input elements like search fields, date pickers, time selectors, and dropdown menus to ensure accurate and consistent data entry. Clearly labeled fields help users understand what information is required, while the organized layout improves usability and reduces errors during data input. Well-designed forms like this play a crucial role in collecting reliable data and enhancing user interaction within the system. Additionally, action buttons such as Save, Save & New, and Cancel provide flexibility and control, making the interface efficient and user-friendly.

3.5 OUTPUT DESIGN

Output design refers to the process of presenting processed information from the system to users in a clear and meaningful format. The main purpose of output design is to ensure that the results produced by the system are easy to understand and useful for decision-making. Outputs can be displayed on screens, reports, dashboards, or notifications depending on user requirements. A well-designed output provides accurate information and helps users quickly interpret the system results.

In the Faculty Day Planner, output design focuses on displaying information such as daily planner schedules, timetable details, attendance reports, and approval status of schedule changes. Faculty members can view their daily activities, lecture timings, and attendance information through the system interface. Administrators and the Head of the Department can also access reports and dashboards to monitor faculty schedules and activities. These outputs help users track academic tasks and maintain organized schedules.

The system outputs are designed to be clear, structured, and user-friendly. Reports and screens include proper headings, labels, and time information so that users can easily understand the displayed data. The system may also generate notifications or confirmation messages when schedules are updated or approvals are completed. Proper output design improves communication between users and the system and ensures that the information generated by the system is effectively utilized.

3.6 ER DIAGRAM

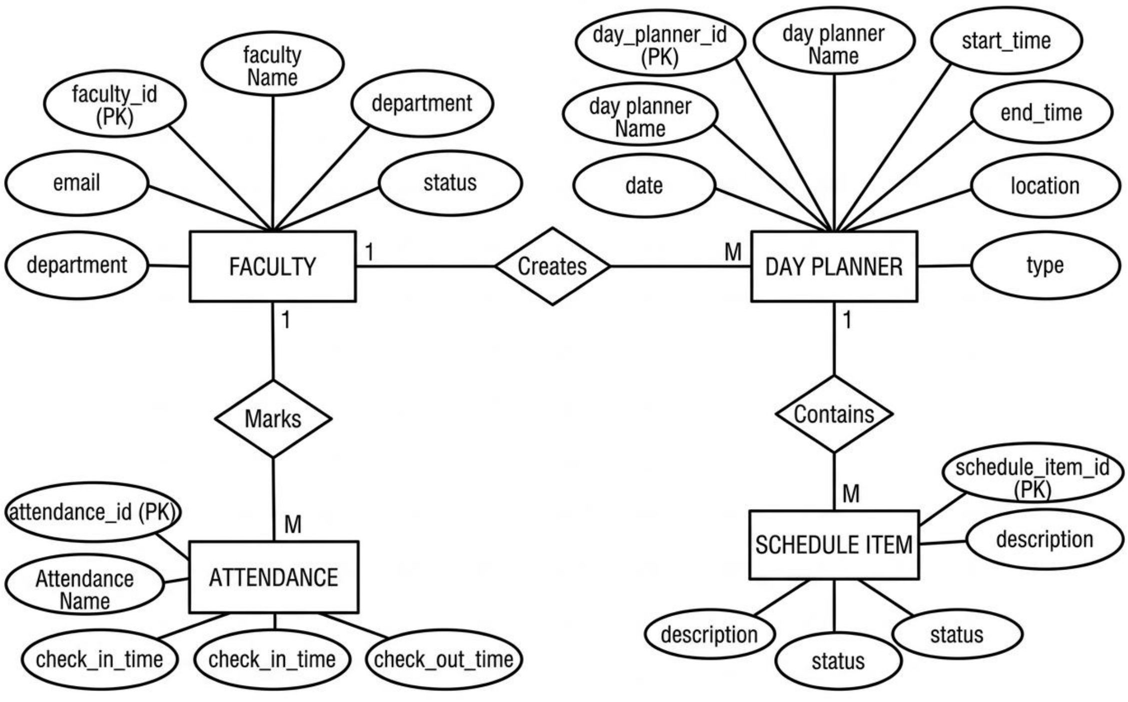

Figure 1: Entity-Relationship Diagram for Faculty Day Planner System

Fig 3.8 ER Diagram

The Entity–Relationship (ER) Diagram for the Faculty Day Planner represents the logical structure of the database and shows how different entities are related to each other. An ER diagram is commonly used in database design to illustrate entities, their attributes, and the relationships between them in a system. Entities are represented by rectangles, attributes by oval shapes, and relationships by diamonds connecting the entities. This model helps in understanding how data is stored and connected within the application before implementing the actual database.

In this diagram, the main entity is FACULTY, which stores information about faculty members such as faculty_id (primary key), faculty name, email, department, and status. A faculty member can create multiple Day Planner records, which establishes a one-to-many (1:M) relationship between FACULTY and DAY PLANNER through the relationship "Creates." The DAY PLANNER entity includes attributes like day_planner_id (primary key), planner name, date, start_time, end_time, location, and type. This entity represents the daily schedule prepared by the faculty for their academic and administrative activities.

The DAY PLANNER entity further contains multiple Schedule Items, forming another one-to-many relationship. The SCHEDULE ITEM entity stores details such as schedule_item_id (primary key), description, and status, representing tasks like lectures, labs, or meetings included in the planner. Additionally, the FACULTY entity is related to the ATTENDANCE entity through the "Marks" relationship, indicating that a faculty member records attendance information. The ATTENDANCE entity contains attributes such as attendance_id (primary key), attendance name, check_in_time, and check_out_time. Together, these entities and relationships define how faculty schedules, attendance records, and daily planning activities are managed within the Faculty Day Planner.

3.7 DATABASE DESIGN

The database design for the Faculty Day Planner is developed to store, manage, and retrieve faculty schedules, day planner records, and attendance information efficiently. A well-structured database ensures that data is organized in tables with proper relationships between entities such as Faculty, Day Planner, Schedule Item, and Attendance. The design follows relational database principles to reduce redundancy and maintain consistency of data. Proper database design also improves system performance and makes it easier to maintain and expand the system in the future.

Data Integration

Data integration refers to the process of combining data from different modules of the system into a unified database structure. In the Faculty Day Planner, information from faculty management, schedule items, day planner, and attendance modules is stored in related tables and connected through primary and foreign keys. This integration allows the system to access and share data efficiently across modules. For example, a faculty member's timetable, daily planner activities, and attendance records can be retrieved together from the database, ensuring better coordination and accurate reporting.

Data Independence

Data independence is an important feature of database systems that allows changes in the database structure without affecting application programs. It means that modifications in the physical storage or logical structure of data do not disturb the system's functioning or user interface. There are two main types of data independence: physical data independence, which allows changes in storage methods without affecting the logical schema, and logical data independence, which allows modifications in tables or relationships without affecting user views or applications. This feature makes the system flexible and easier to maintain.

Data Security

Data security ensures that the information stored in the database is protected from unauthorized access, misuse, or modification. In the Faculty Day Planner, security measures such as user authentication, role-based access control, and data validation are implemented to protect sensitive information. For example, faculty members can access and update their own schedules, while the Head of Department has permission to approve or modify schedule changes. These security mechanisms help maintain confidentiality, integrity, and reliability of the stored data.

3.8 TABLE DESIGN

In a database system, tables are used to store data in the form of rows and columns. Each column is called a field, and every field has a data type that defines what type of data it can store such as text, numbers, or dates. Proper table design helps maintain data accuracy, consistency, and efficient retrieval of information in the system.

Table 3.1 Faculty Table

Field Name	Data Type	Description
faculty_id (PK)	INT	Unique identifier for each faculty member
faculty_name	VARCHAR(100)	Name of the faculty member
email	VARCHAR(100)	Official email address of the faculty
department	VARCHAR(100)	Department to which the faculty belongs
status	VARCHAR(20)	Indicates whether the faculty is active or inactive

The Faculty table stores the basic information of all faculty members who use the Faculty Day Planner. Each record in this table represents one faculty member and contains details such as faculty ID, name, email, department, and status. The faculty_id acts as the primary key and uniquely identifies each faculty record in the database. This table is important because it connects faculty members with their day planners and attendance records. In relational databases, each row represents a unique record while columns represent specific attributes of that entity.

Table 3.2 Day Planner Table

Field Name	Data Type	Description
day_planner_id (PK)	INT	Unique identifier for each day planner
planner_name	VARCHAR(100)	Name or title of the day planner
date	DATE	Date for which the planner is created
start_time	TIME	Starting time of the scheduled activity
end_time	TIME	Ending time of the scheduled activity
location	VARCHAR(100)	Location where the activity takes place
type	VARCHAR(50)	Type of activity (Lecture, Lab, Meeting, etc.)
faculty_id (FK)	INT	References the faculty who created the planner

The Day Planner table stores information about the daily schedule created by faculty members. It contains details such as planner ID, planner name, date, start time, end time, location, and activity type. The day_planner_id serves as the primary key and uniquely identifies each planner entry. This table helps organize daily academic tasks like lectures, labs, meetings, and other activities for faculty members. Each day planner is associated with a faculty member, allowing the system to manage and retrieve daily schedules efficiently.

Table 3.3 Schedule Item Table

Field Name	Data Type	Description
schedule_item_id (PK)	INT	Unique identifier for each schedule item
description	VARCHAR(255)	Description of the schedule activity
status	VARCHAR(20)	Status of the schedule (Pending, Approved, Completed)
day_planner_id (FK)	INT	References the day planner that contains the schedule item

The Schedule Item table stores individual activities or tasks that are included within a day planner. Each record represents a specific schedule item such as a lecture, lab session, or meeting. The table contains fields like schedule item ID, description, and status of the activity. The schedule_item_id is the primary key that uniquely identifies each schedule entry. This table allows the system to break down a day planner into multiple manageable activities, making it easier to track and update daily schedules.

Table 3.4 Attendance Table

Field Name	Data Type	Description
attendance_id (PK)	INT	Unique identifier for each attendance record
attendance_name	VARCHAR(100)	Name or type of attendance record
check_in_time	TIME	Time when faculty checked in
check_out_time	TIME	Time when faculty checked out
faculty_id (FK)	INT	References the faculty who marked attendance

The Attendance table stores attendance information recorded by faculty members. It contains details such as attendance ID, attendance name, check-in time, and check-out time. The attendance_id acts as the primary key and uniquely identifies each attendance record in the database. This table helps track faculty attendance and working hours for reporting and monitoring purposes. By linking attendance records with faculty details, the system can generate attendance reports and maintain accurate records of faculty presence.

SUMMARY

This chapter explains the system design of the Faculty Day Planner and how the system components are organized. It describes the different modules such as faculty management, schedule management, day planner, attendance, notifications, and weekly schedule maintenance. The chapter also includes design diagrams such as DFD Level-0, DFD Level-1, System Flow Diagram, and ER Diagram to represent the system structure and data flow. In addition, the chapter discusses input design, output design, database design, and table structures used to store system data. Overall, the system design acts as a blueprint for developing and implementing the Faculty Day Planner application.

CHAPTER 4

IMPLEMENTATION

4.1 CODE DESCRIPTION

The Code Description section explains how the system is implemented through programming logic and modules. It describes how different parts of the application work together to perform the required functions. Instead of showing the entire source code, this section usually explains the purpose of important modules, functions, and processes used in the system. This helps readers understand how the system is developed and how each component contributes to the overall functionality.

In the Faculty Day Planner, the code is organized into different modules such as Faculty Management, Day Planner Management, Schedule Item Management, Attendance Management, and Notification Handling. Each module performs specific tasks like creating faculty records, generating daily planners from timetable data, updating schedule items, recording attendance, and sending notifications for schedule changes. The program logic ensures that data entered through the user interface is validated and stored correctly in the database. This modular coding structure makes the system easier to maintain and update.

The implementation also includes database operations such as create, read, update, and delete (CRUD) for managing system data. These operations allow users to add new records, view stored information, update existing data, and remove unnecessary records when required. Error handling and validation are also included in the code to ensure that invalid data is not stored in the system. Overall, the code description explains how the developed program supports the system functionalities and ensures smooth operation of the Faculty Day Planner application.

4.2 Standardization of the Coding

Standardization of coding refers to the use of predefined rules and guidelines while writing program code. These standards ensure that the code written by different developers follows a uniform structure and style. Coding standards include rules for naming variables, formatting code, adding comments, and organizing program modules.

Following these guidelines helps maintain consistency and improves the quality of the software system.

In the Faculty Day Planner, coding standards are followed to maintain a clear and structured program design. Meaningful variable names are used for objects such as Faculty, Day Planner, Schedule Item, and Attendance. Proper indentation, modular coding, and comments are included so that the code can be easily understood by other developers. Each module of the system is written separately to perform specific tasks such as managing schedules, recording attendance, and generating reports.

By maintaining coding standards, the system becomes easier to maintain, debug, and update in the future. Standardized code improves readability and helps developers quickly identify errors or make modifications when new features are added. It also supports teamwork because all developers follow the same programming practices. Therefore, coding standardization ensures better reliability, maintainability, and overall quality of the Faculty Day Planner.

4.3 Error Handling

Error handling is the process of identifying, managing, and responding to errors that occur during the execution of a program. Errors may occur due to incorrect user input, system failures, or unexpected program conditions. Proper error handling helps prevent the system from crashing and ensures that the application continues to operate in a stable manner. It improves the reliability and robustness of the software system.

In the Faculty Day Planner, error handling mechanisms are implemented to detect and manage errors that may occur during data entry or system processing. Input validation is used to check whether users enter correct information such as valid email formats, required fields, and proper date or time values. If an error occurs, the system displays appropriate error messages to guide the user in correcting the mistake. This helps maintain data accuracy and prevents invalid information from being stored in the database.

The system also includes exception handling and validation checks to manage runtime errors and system failures. When an unexpected error occurs, the system records the error and displays a meaningful message instead of stopping the program abruptly. Proper error handling ensures smooth system operation and improves user experience by helping users quickly identify and correct problems.

SUMMARY

This chapter describes the implementation phase of the Faculty Day Planner and explains how the system is developed through coding and programming. It discusses the structure of the program modules and how the system functions are implemented using proper coding techniques. The chapter also explains the standardization of coding, which ensures that the program follows consistent coding rules and improves readability and maintainability. In addition, error handling mechanisms are included to detect and manage system errors during execution. Overall, the implementation phase focuses on converting the system design into a working software application.

CHAPTER 5

TESTING AND RESULTS

5.1 TESTING

Testing is an important phase in software development that ensures the system works correctly and meets the specified requirements. It is the process of executing a program or system to identify errors, bugs, or missing functions. The main objective of testing is to verify that the system performs its tasks accurately and provides correct outputs for given inputs. Proper testing improves the reliability, performance, and quality of the software system.

In the Faculty Day Planner, testing is performed to verify that all modules such as faculty management, schedule items, day planner creation, attendance recording, and notification functions operate correctly. The system is tested using different input values to ensure that data is stored and processed properly. Testing also checks whether approval processes, notifications, and weekly schedule updates work as expected. This helps ensure that the system functions smoothly without errors.

Various types of testing such as unit testing, integration testing, and system testing are performed during the development process. These tests verify individual modules, interactions between modules, and the complete system functionality. System testing evaluates the entire integrated application to confirm that it satisfies all user requirements before deployment.

5.1.1 Unit Testing

Unit testing is a software testing technique where individual modules or components of a system are tested separately to verify that they work correctly. Each test case checks the module by giving specific inputs and comparing the expected output with the actual output. This helps identify errors early in the development stage and ensures that each part of the system functions properly.

CASE 1

Module: Faculty Management Module

Test Type: Data Entry Verification

Input: Faculty Name, Email, Department, Status

Output: Faculty Record Created Successfully

TEST

Input: Faculty Name = “John”, Email = “john@college.edu”, Department = “CSE”, Status = Active

Output: Faculty record saved and displayed in faculty list

Analysis: The actual output matches the expected output.

Result: Pass

CASE 2

Module: Schedule Item (Timetable) Module

Test Type: Timetable Creation Verification

Input: Day, Time, Subject, Schedule Type

Output: Schedule item added to timetable

TEST

Input: Monday, 9:00–10:00 AM, DBMS, Lecture

Output: Schedule item stored and displayed in timetable

Analysis: The system successfully stored and displayed the timetable data.

Result: Pass

CASE 3

Module: Day Planner Module

Test Type: Day Planner Generation

Input: Faculty ID and Date

Output: Day planner created with schedule items

TEST

Input: Faculty ID = F001, Date = 15-03-2026

Output: Day planner generated showing lecture schedule for that day

Analysis: The system correctly generated the day planner based on the timetable.

Result: Pass

CASE 4

Module: Schedule Change Request & Approval Module

Test Type: Approval Process Verification

Input: Schedule change request submitted by faculty

Output: Request sent to HOD for approval

TEST

Input: Change lecture time from 10:00 AM to 11:00 AM

Output: Request status updated as "Pending Approval"

Analysis: The system correctly sent the request to HOD and updated the status.

Result: Pass

CASE 5

Module: Notification Module

Test Type: Notification Delivery

Input: Schedule updated by HOD

Output: Notification sent to faculty email

TEST

Input: HOD approves schedule change

Output: Faculty receives notification message

Analysis: The notification was successfully sent to the faculty member.

Result: Pass

5.1.2 Integration Testing

Integration testing verifies whether different modules of the system work correctly when combined together. It focuses on checking the interaction and data flow between modules such as Faculty, Day Planner, Schedule Item, and Notification modules. This testing is performed after unit testing to ensure that the integrated components communicate properly and perform their tasks correctly.

CASE 1

Modules: Faculty Module + Day Planner Module

Test Type: Data Integration

Input: Faculty record created

Output: Faculty appears in Day Planner selection list

TEST

Input: Add new faculty "John – CSE"

Output: Faculty name available while creating day planner

Analysis: Faculty data correctly integrated with the day planner module.

Result: Pass

CASE 2

Modules: Schedule Item Module + Day Planner Module

Test Type: Schedule Generation

Input: Timetable schedule item created

Output: Day planner automatically displays schedule for that day

TEST

Input: Monday 9:00–10:00 Lecture added in timetable

Output: Monday day planner shows lecture

Analysis: Schedule item successfully integrated with day planner.

Result: Pass

CASE 3

Modules: Schedule Change Request Module + Approval Module

Test Type: Approval Workflow

Input: Faculty submits schedule change request

Output: Request visible to HOD for approval

TEST

Input: Change lecture time request submitted

Output: HOD approval screen shows pending request

Analysis: Request correctly transferred between modules.

Result: Pass

CASE 4

Modules: Approval Module + Notification Module

Test Type: Notification Trigger

Input: HOD approves schedule change

Output: Notification sent to faculty

TEST

Input: HOD clicks approve

Output: Notification message generated

Analysis: Notification triggered successfully after approval.

Result: Pass

CASE 5

Modules: Attendance Module + Faculty Module

Test Type: Data Linking

Input: Faculty marks attendance

Output: Attendance stored under faculty record

TEST

Input: Faculty check-in time recorded

Output: Attendance record displayed for that faculty

Analysis: Attendance correctly linked with faculty information.

Result: Pass

5.1.3 Validation Testing

Validation testing ensures that the entire system works according to user requirements and meets expected functionality. It confirms that the developed system satisfies user needs and performs all required operations correctly.

CASE 1

Module: Faculty Day Planner

Test Type: Functional Validation

Input: Faculty logs in and views daily planner

Output: Planner schedule displayed correctly

TEST

Input: Faculty login credentials

Output: Day planner page opened

Analysis: System displays correct schedule information.

Result: Pass

CASE 2

Module: Schedule Change System

Test Type: Requirement Validation

Input: Faculty requests schedule change

Output: Request sent for HOD approval

TEST

Input: Change request submitted

Output: Request status updated to “Pending”

Analysis: System performs requested functionality correctly.

Result: Pass

CASE 3

Module: Weekly Schedule Maintenance

Test Type: Functional Validation

Input: Weekly tasks other than lecture/lab

Output: Tasks automatically removed on Monday

TEST

Input: Meeting task added previous week

Output: Task removed on Monday

Analysis: System correctly performs weekly cleanup.

Result: Pass

CASE 4

Module: Notification System

Test Type: User Requirement Validation

Input: Schedule updated by HOD

Output: Faculty receives notification

TEST

Input: HOD modifies timetable

Output: Faculty receives update message

Analysis: Notification feature works as expected.

Result: Pass

CASE 5

Module: Reports and Dashboard

Test Type: Output Validation

Input: Generate faculty schedule report

Output: Report displayed with correct data

TEST

Input: Select faculty and generate report

Output: Timetable report displayed

Analysis: System generates accurate reports.

Result: Pass

SUMMARY

This chapter discussed the testing phase of the Faculty Day Planner, which ensures that the developed system works correctly and meets user requirements. Various testing techniques such as unit testing, integration testing, and validation testing were performed to verify the functionality of different modules in the system. Each module was tested with different inputs and the outputs were compared with the expected results to identify possible errors. Software testing helps detect defects and ensure that the system performs reliably before deployment. Overall, the testing results showed that the Faculty Day Planner operates correctly and all modules function as expected. The successful test cases confirm that the system can manage faculty schedules, attendance, and notifications efficiently without major errors. Thus, the testing process ensures the quality, reliability, and proper performance of the developed application.

CHAPTER 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Faculty Day Planner was developed to improve the management of faculty schedules, daily activities, and attendance in a college environment. The system provides a structured platform where faculty members can view their timetable, manage daily tasks, and request schedule changes when required. The HOD can approve or modify schedule requests, and the system automatically notifies faculty members about any updates. This improves communication and coordination between faculty and administration.

The project successfully integrates different modules such as faculty management, schedule items, day planner generation, attendance management, notifications, and reporting. These modules work together to automate the process of timetable management and daily planning. The system also ensures data accuracy and efficiency through validation, testing, and proper database design. Software projects are considered successful when they effectively meet user requirements and system functionality goals.

Overall, the Faculty Day Planner simplifies the process of managing faculty schedules and improves productivity within the institution. The application provides an organized and user-friendly solution for handling academic activities and schedule changes. By implementing this system, colleges can reduce manual work, minimize scheduling conflicts, and maintain accurate records of faculty activities. The project demonstrates how modern digital systems can enhance the efficiency of academic administration.

6.2 Future Enhancements

The Faculty Day Planner can be further improved by adding advanced features and technologies in the future. One possible enhancement is the development of a mobile application so that faculty members can access their schedules, update tasks, and receive notifications through smartphones anytime and anywhere. Mobile-friendly interfaces are becoming an important feature of modern academic management systems because they allow users to manage schedules and communicate easily on the go.

Another future improvement is the integration of Artificial Intelligence (AI) for smart timetable generation and schedule optimization. AI-based systems can automatically generate conflict-free timetables by analyzing faculty availability, classroom resources, and workload distribution. These systems can also update schedules in real time when changes occur, improving efficiency and reducing manual effort.

The system can also be enhanced by integrating additional institutional systems such as Learning Management Systems (LMS), biometric attendance systems, and advanced analytics dashboards. Such integrations would allow automatic attendance tracking, improved communication, and data-driven decision-making for administrators. Advanced analytics can generate insights and reports that help institutions plan academic activities more effectively.

Finally, future versions of the system can include cloud-based deployment and enhanced security features such as encryption, multi-factor authentication, and role-based access control. These improvements will ensure better data protection, scalability, and accessibility for colleges. With these enhancements, the Faculty Day Planner can evolve into a complete digital solution for academic schedule and faculty management.

APPENDICES

A.SAMPLE CODING

BACKEND

faculty__c.object-meta.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<CustomObject xmlns="http://soap.sforce.com/2006/04/metadata">
    <actionOverrides>
        <actionName>Accept</actionName>
        <type>Default</type>
    </actionOverrides>
    <actionOverrides>
        <actionName>Accept</actionName>
        <formFactor>Large</formFactor>
        <type>Default</type>
    </actionOverrides>
    <actionOverrides>
        <actionName>Accept</actionName>
        <formFactor>Small</formFactor>
        <type>Default</type>
    </actionOverrides>
    <actionOverrides>
        <actionName>CancelEdit</actionName>
        <type>Default</type>
    </actionOverrides>
    <actionOverrides>
        <actionName>CancelEdit</actionName>
        <formFactor>Large</formFactor>
        <type>Default</type>
    </actionOverrides>
    <actionOverrides>
        <actionName>CancelEdit</actionName>
        <formFactor>Small</formFactor>
        <type>Default</type>
```

```xml
</actionOverrides>
<actionOverrides>
    <actionName>Clone</actionName>
    <type>Default</type>
</actionOverrides>
<actionOverrides>
    <actionName>Clone</actionName>
    <formFactor>Large</formFactor>
    <type>Default</type>
</actionOverrides>
<actionOverrides>
    <actionName>Clone</actionName>
    <formFactor>Small</formFactor>
    <type>Default</type>
</actionOverrides>
<actionOverrides>
    <actionName>Delete</actionName>
    <type>Default</type>
</actionOverrides>
<actionOverrides>
    <actionName>Delete</actionName>
    <formFactor>Large</formFactor>
    <type>Default</type>
</actionOverrides>
<actionOverrides>
    <actionName>Delete</actionName>
    <formFactor>Small</formFactor>
    <type>Default</type>
</actionOverrides>
<actionOverrides>
    <actionName>Edit</actionName>
    <type>Default</type>
</actionOverrides>
```

```
<actionOverrides>
   <actionName>Edit</actionName>
   <formFactor>Large</formFactor>
   <type>Default</type>
</actionOverrides>
<actionOverrides>
   <actionName>Edit</actionName>
   <formFactor>Small</formFactor>
   <type>Default</type>
</actionOverrides>
<actionOverrides>
   <actionName>List</actionName>
   <type>Default</type>
</actionOverrides>
<actionOverrides>
   <actionName>List</actionName>
   <formFactor>Large</formFactor>
   <type>Default</type>
</actionOverrides>
<actionOverrides>
   <actionName>List</actionName>
   <formFactor>Small</formFactor>
   <type>Default</type>
</actionOverrides>
<actionOverrides>
   <actionName>New</actionName>
   <type>Default</type>
</actionOverrides>
<actionOverrides>
   <actionName>New</actionName>
   <formFactor>Large</formFactor>
   <type>Default</type>
</actionOverrides>
```

```xml
<actionOverrides>
   <actionName>New</actionName>
   <formFactor>Small</formFactor>
   <type>Default</type>
</actionOverrides>
<actionOverrides>
   <actionName>SaveEdit</actionName>
   <type>Default</type>
</actionOverrides>
<actionOverrides>
   <actionName>SaveEdit</actionName>
   <formFactor>Large</formFactor>
   <type>Default</type>
</actionOverrides>
<actionOverrides>
   <actionName>SaveEdit</actionName>
   <formFactor>Small</formFactor>
   <type>Default</type>
</actionOverrides>
<actionOverrides>
   <actionName>Tab</actionName>
   <type>Default</type>
</actionOverrides>
<actionOverrides>
   <actionName>Tab</actionName>
   <formFactor>Large</formFactor>
   <type>Default</type>
</actionOverrides>
<actionOverrides>
   <actionName>Tab</actionName>
   <formFactor>Small</formFactor>
   <type>Default</type>
</actionOverrides>
```

```
<actionOverrides>
    <actionName>View</actionName>
    <type>Default</type>
</actionOverrides>
<actionOverrides>
    <actionName>View</actionName>
    <formFactor>Large</formFactor>
    <type>Default</type>
</actionOverrides>
<actionOverrides>
    <actionName>View</actionName>
    <formFactor>Small</formFactor>
    <type>Default</type>
</actionOverrides>
<allowInChatterGroups>false</allowInChatterGroups>
<compactLayoutAssignment>SYSTEM</compactLayoutAssignment>
<deploymentStatus>Deployed</deploymentStatus>
<enableActivities>true</enableActivities>
<enableBulkApi>true</enableBulkApi>
<enableFeeds>false</enableFeeds>
<enableHistory>true</enableHistory>
<enableLicensing>false</enableLicensing>
<enableReports>true</enableReports>
<enableSearch>false</enableSearch>
<enableSharing>true</enableSharing>
<enableStreamingApi>true</enableStreamingApi>
<externalSharingModel>Private</externalSharingModel>
<label>faculty</label>
<nameField>
    <label>faculty Name</label>
    <trackHistory>false</trackHistory>
    <type>Text</type>
</nameField>
```

```
    <pluralLabel>faculties</pluralLabel>
    <searchLayouts></searchLayouts>
    <sharingModel>ReadWrite</sharingModel>
    <visibility>Public</visibility>
</CustomObject>
```


FRONTEND


```
<?xml version="1.0" encoding="UTF-8"?>
<Flow xmlns="http://soap.sforce.com/2006/04/metadata">
    <actionCalls>
        <name>Notify_faculty</name>
        <label>Notify faculty</label>
        <locationX>0</locationX>
        <locationY>0</locationY>
        <actionName>emailSimple</actionName>
        <actionType>emailSimple</actionType>
        <flowTransactionModel>CurrentTransaction</flowTransactionModel>
        <inputParameters>
            <name>recipientAddresses</name>
            <value/>
        </inputParameters>
        <inputParameters>
            <name>senderAddress</name>
        </inputParameters>
        <inputParameters>
            <name>composeEmailContent</name>
            <value>
                <stringValue>True</stringValue>
            </value>
        </inputParameters>
        <inputParameters>
            <name>emailSubject</name>
            <value>
                <stringValue>Schedule Changes</stringValue>
```

```
            </value>
        </inputParameters>
        <inputParameters>
            <name>emailBody</name>
            <value>
                <stringValue><p>Hi {!$Record.faculty__r.Name}!!</p><p>Your Day
Planner has been created or updated.</p><p><br></p><p>Please login to the system to
view the schedule.</p><p><br></p><p>Thank you.</p></stringValue>
            </value>
        </inputParameters>
        <nameSegment>emailSimple</nameSegment>
    </actionCalls>
    <apiVersion>66.0</apiVersion>
    <environments>Default</environments>
    <interviewLabel>hod schedules {!$Flow.CurrentDateTime}</interviewLabel>
    <label>hod schedules</label>
    <processMetadataValues>
        <name>BuilderType</name>
        <value>
            <stringValue>LightningFlowBuilder</stringValue>
        </value>
    </processMetadataValues>
    <processMetadataValues>
        <name>CanvasMode</name>
        <value>
            <stringValue>AUTO_LAYOUT_CANVAS</stringValue>
        </value>
    </processMetadataValues>
    <processMetadataValues>
        <name>OriginBuilderType</name>
        <value>
            <stringValue>LightningFlowBuilder</stringValue>
        </value>
```

```
    </processMetadataValues>
    <processType>AutoLaunchedFlow</processType>
    <recordLookups>
        <name>get_faculty_mail</name>
        <label>get faculty mail</label>
        <locationX>0</locationX>
        <locationY>0</locationY>

<assignNullValuesIfNoRecordsFound>false</assignNullValuesIfNoRecordsFound>
        <connector>
            <targetReference>Notify_faculty</targetReference>
        </connector>
        <filterLogic>and</filterLogic>
        <filters>
            <field>Id</field>
            <operator>EqualTo</operator>
            <value>
                <elementReference>$Record.faculty__c</elementReference>
            </value>
        </filters>
        <getFirstRecordOnly>true</getFirstRecordOnly>
        <object>faculty__c</object>
        <storeOutputAutomatically>true</storeOutputAutomatically>
    </recordLookups>
    <start>
        <locationX>0</locationX>
        <locationY>0</locationY>
        <connector>
            <targetReference>get_faculty_mail</targetReference>
        </connector>
        <filterLogic>and</filterLogic>
        <filters>
            <field>status__c</field>
```

```
            <operator>EqualTo</operator>
            <value>
                <stringValue>Approved</stringValue>
            </value>
        </filters>
        <object>day_planner__c</object>
        <recordTriggerType>CreateAndUpdate</recordTriggerType>
        <triggerType>RecordAfterSave</triggerType>
    </start>
    <status>Active</status>
</Flow>
```

B.SCREENSHOTS

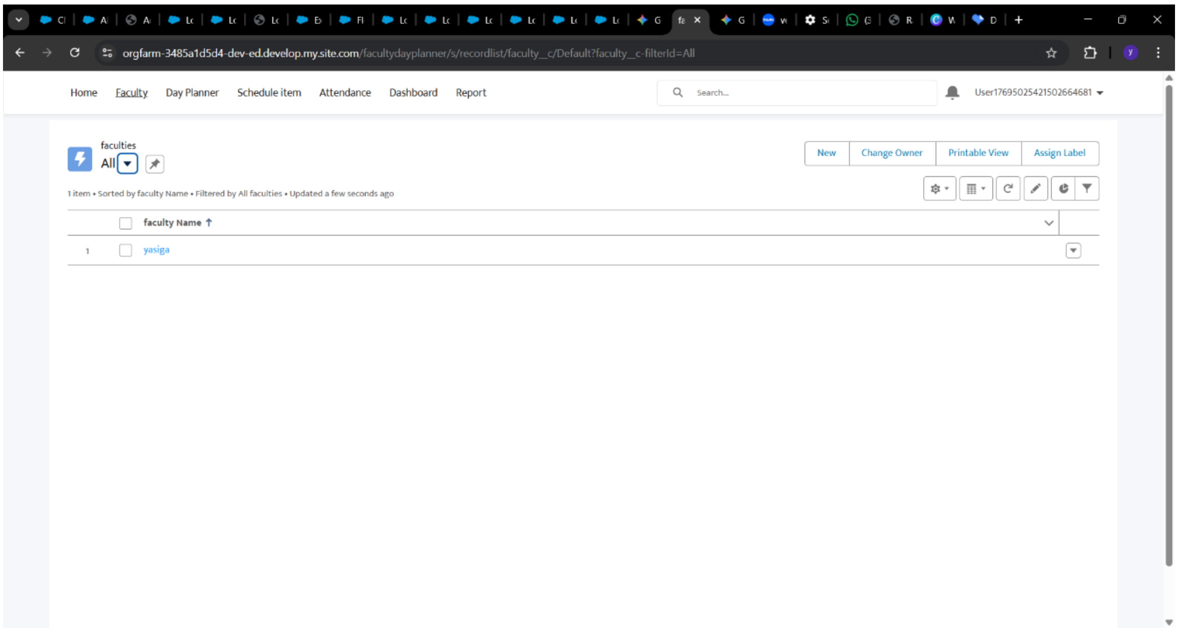

Figure B.1 Faculty Data

This screen displays the list of all faculty records available in the system. Users can view, search, and manage faculty details through this interface. It provides quick access to create new faculty and maintain faculty information efficiently.

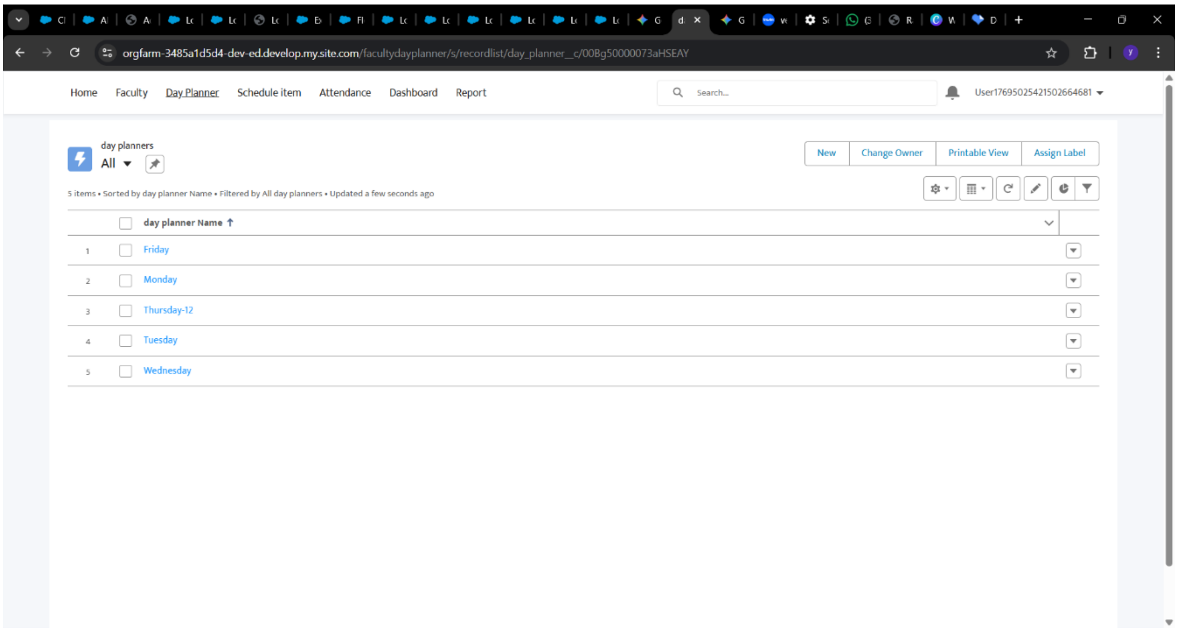

Figure B.2 Day Planner Data

This screen shows all created day planners for different days. Users can view and manage daily schedules associated with each planner. It helps in organizing and accessing faculty activities for specific days.

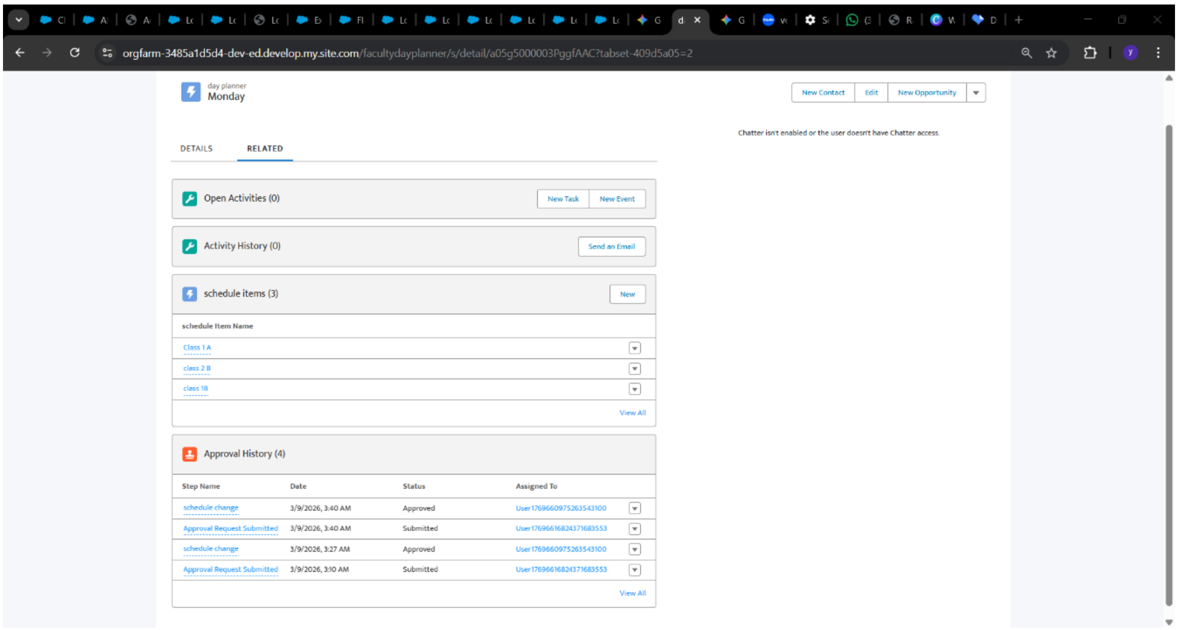

Figure B.3 Scheduled Item

This screen displays detailed information of a selected day planner. It includes related schedule items and approval history for tracking changes. Users can manage activities and monitor approval status within a single view.

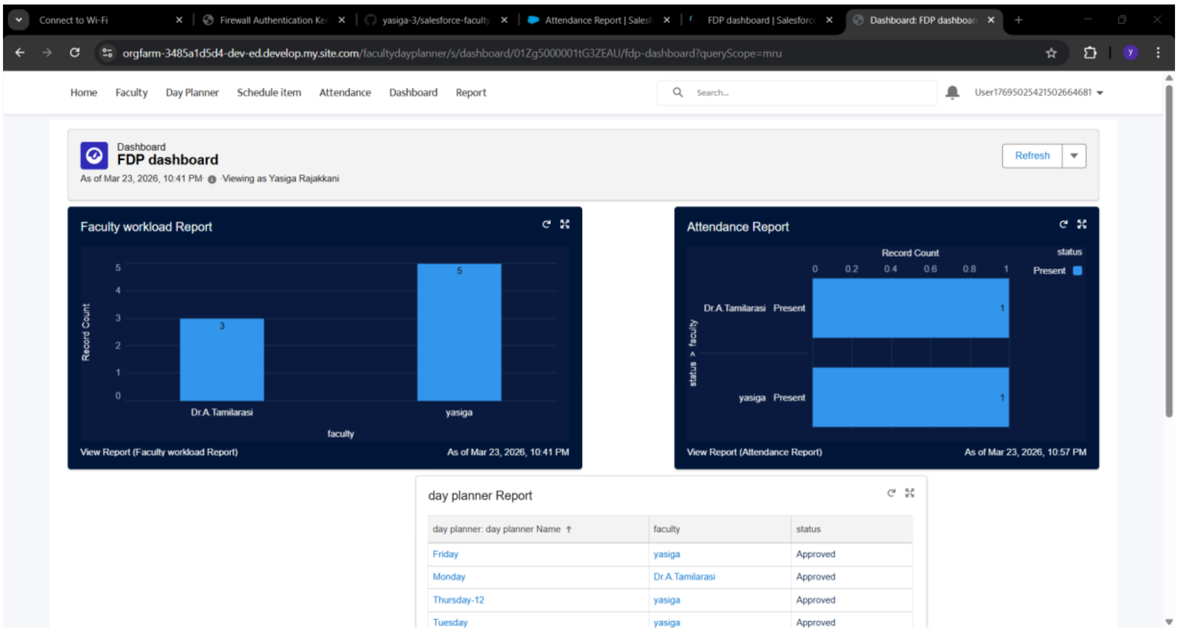

Figure B.4 Dashboard

This screen presents visual reports such as charts and tables for faculty workload and attendance. Dashboards provide a graphical representation of key data for easy analysis and decision-making. It helps users quickly understand system performance and track important metrics.

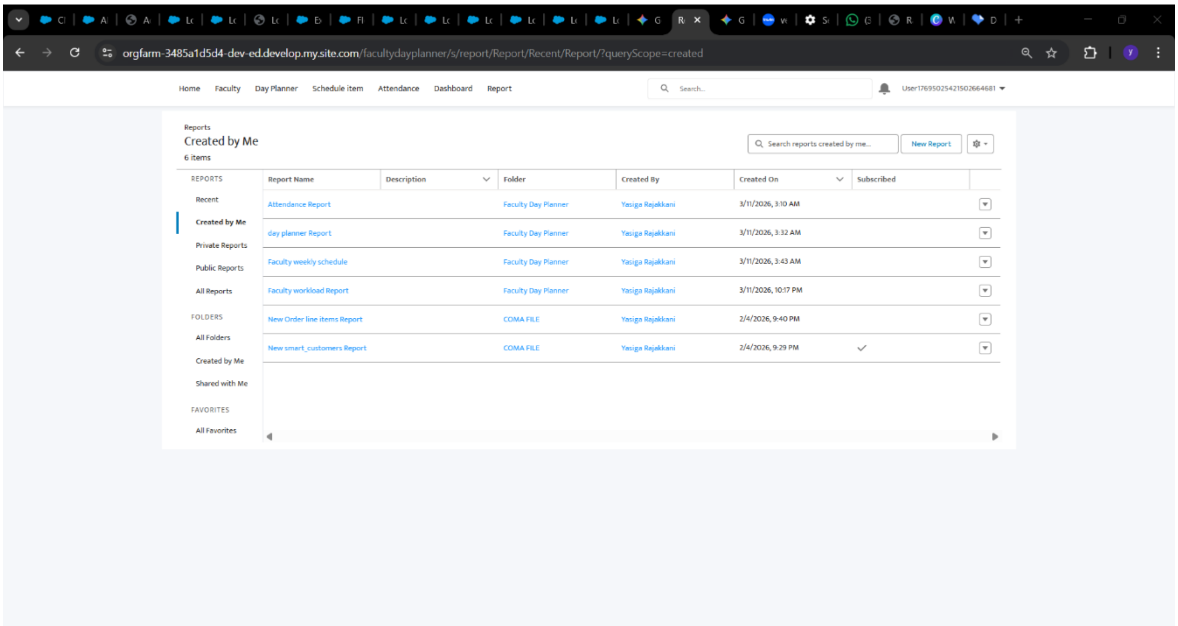

Figure B.5 Report

This screen shows a list of generated reports created by the user. Users can view, organize, and access different reports related to faculty, schedules, and attendance. Reports display structured data in tabular format for detailed analysis and record keeping

REFERENCES

WEB REFERENCES

1. https://developer.salesforce.com
2. https://trailhead.salesforce.com
3. https://www.geeksforgeeks.org/software-engineering
4. https://www.salesforceben.com
5. https://www.tutorialspoint.com/salesforce

These websites were used to understand Salesforce development, system design concepts, and software engineering techniques.

BOOK & JOURNAL REFERENCES

1. Roger S. Pressman – Software Engineering: A Practitioner's Approach
2. Ian Sommerville – Software Engineering
3. David Liu – Beginning Salesforce Development with Apex
4. Jason Ouellette – Salesforce Platform App Builder Certification Guide

These books provided theoretical concepts related to system analysis, software development, and Salesforce implementation.

SUSTAINABLE DEVELOPMENT GOALS (SDGs)

1. SDG 4 – Quality Education

This goal focuses on ensuring inclusive and equitable quality education and promoting lifelong learning opportunities for all.

The Faculty Day Planner supports this goal by helping faculty manage schedules efficiently, improving teaching organization, and enhancing academic productivity. It ensures better planning of lectures, labs, and academic activities, which directly improves the quality of education delivered to students.

2. SDG 9 – Industry, Innovation and Infrastructure

This goal aims to build resilient infrastructure and promote innovation through technology.

This project uses a cloud-based platform (Salesforce) to digitize academic scheduling, replacing manual processes with a smart system. This promotes innovation in colleges and improves operational efficiency through modern digital infrastructure.

3. SDG 8 – Decent Work and Economic Growth

This goal promotes productive work environments and efficient work management.

This project helps faculty manage workload, schedules, and attendance effectively, reducing stress and improving work productivity. It ensures better time management and supports a structured and efficient working environment in colleges.